COMANDOS BÁSICOS DE LINUX

Implantación de Sistemas Operativos
1º ASIR

Autor: Enrique Laín

Consola de Ubuntu. Introducción

Como ocurre en Unix, Linux ofrece interfaces en modo texto llamados consolas o terminales virtuales. Consiste en que a partir de una entrada estándar (el teclado) y con una salida estándar (el monitor) se simulen varias terminales, pudiendo conectarse varios usuarios simultáneamente (ya que Línux es un SO multiusuario).

El cambio de una consola a otra se realiza a través de la combinación de teclas **Ctrl+Alt+Fx** (las teclas de Función: F1 hasta F6). Donde **Ctrl+Alt+F7** muestra una consola gráfica.

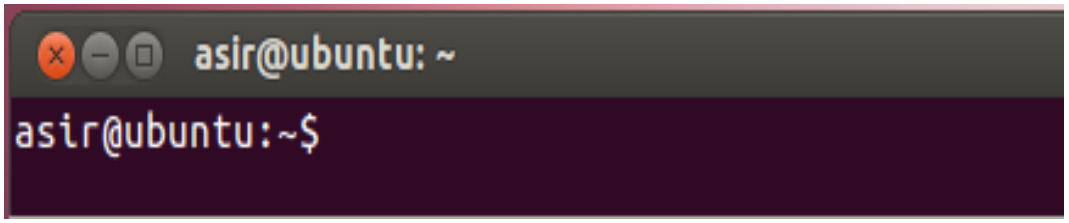
NOTAS:

*Si estamos trabajando con VirtualBox utilizaremos **Ctrl+Fx** y para la consola gráfica **Ctrl+F7**.*

*En VMWare habría que reconfigurar el fichero **preferencias** para cambiar la combinación de teclas que permiten acceder y cambiar de consola, debido a que Ctrl+Alt en Vmware permite liberar el ratón.*

PROMPT DEL SISTEMA

- Este entorno de texto donde nos encontramos y que nos permite introducir comandos se conoce como **Shell**, siendo capaz de interpretar comandos, sentencias y lenguaje de programación mediante scripts.
- El Shell que más se usa es conocido como **bash**, aunque existen otros: csh, ksh, etc.
- Cuando arrancamos una sesión con un usuario en una máquina concreta, al abrir una consola veremos algo similar a lo siguiente:



Donde:

asir es el usuario con el que iniciamos sesión.
ubuntu es el nombre que se le dió a la máquina al instalarla.

~ indica que estamos en el directorio home/asir

$ indica que estamos con un usuario normal, no root.

en el caso de que estuviéramos como superusuario **root** (usuario administrador que goza de todos los privilegios para administrar y gestionar el sistema)

MOSTRAR AYUDA SOBRE COMANDOS

Desde el shell podemos solicitar ayuda sobre algún comando concreto de las siguientes maneras:

nombre_comando - -help	**ejemplo: mkdir --help**
info nombre_comando	**ejemplo: info cat**
man nombre_comando	**ejemplo: man mkdir**
apropos nombre_comando	**ejemplo: apropos mkdir**

NOTAS:

man muestra un manual completo sobre algún comando o bien sobre todos ellos si tecleamos únicamente man.

Con el comando **apropos**, dada una palabra busca los comandos relacionados sobre ella.

COMANDOS PARA FICHEROS Y DIRECTORIOS

COMANDO ls

El comando **ls** permite listar el contenido de un directorio.
Sintaxis: ls [opciones] [directorio]

Algunas opciones:
-l : muestra la salida en formato largo.
-R: lista recursivamente un directorio.
-a : lista además los ficheros ocultos (sus nombres comienzan con punto).
-h : muestra el tamaño de los ficheros en forma más legible (Ej.: 16M, 4k, etc.)
-i : muestra el identificador del i-nodo asociado a cada elemento

Ejemplos:

$ ls -hl /etc # Muestra el contenido del directorio **/etc** en formato largo y el tamaño de cada directorio y fichero.
$ ls -R /usr # Muestra de forma recursiva cada contenido de cada directorio de **/usr**.
$ ls -al # Muestra el contenido del directorio **actual** en formato largo y ficheros ocultos..
$ ls -ali /etc # Muestra el contenido del directorio **/etc** en formato largo, ficheros ocultos y el i-nodo de cada uno.

COMANDOS PARA FICHEROS Y DIRECTORIOS

COMANDO cd

El comando cd permite cambiar de directorio.
Sintaxis: cd [directorio]

cd .. # Sube un nivel en el árbol de directorios
cd # Nos lleva al directorio **home** del usuario.
cd nombre_directorio # Cambia al directorio indicado.
cd ~ **#** Cambia al directorio home
cd / **#** Nos lleva al directorio raiz /

Ejemplos:

$ cd /etc # Cambia al directorio /etc
$ cd # Cambia de directorio y nos deja en el directorio base del usuario actual
$ cd ~ **(alt + 126)** # Cambia al directorio base (home) del usuario actual. (El símbolo ~ se entiende en bash como el directorio home del usuario).

COMANDOS PARA FICHEROS Y DIRECTORIOS

COMANDO pwd

pwd indica el camino absoluto del directorio en el que nos encontramos actualmente.

Ejemplos:

```
henry@ubuntu: ~
henry@ubuntu:~$ pwd
/home/henry
henry@ubuntu:~$
```

```
henry@ubuntu: ~/Descargas
henry@ubuntu:~/Descargas$ pwd
/home/henry/Descargas
henry@ubuntu:~/Descargas$
```

COMANDOS PARA FICHEROS Y DIRECTORIOS

COMANDO mkdir

mkdir se utiliza para crear directorios.

Sintaxis: **mkdir [directorio]**

Ejemplos:

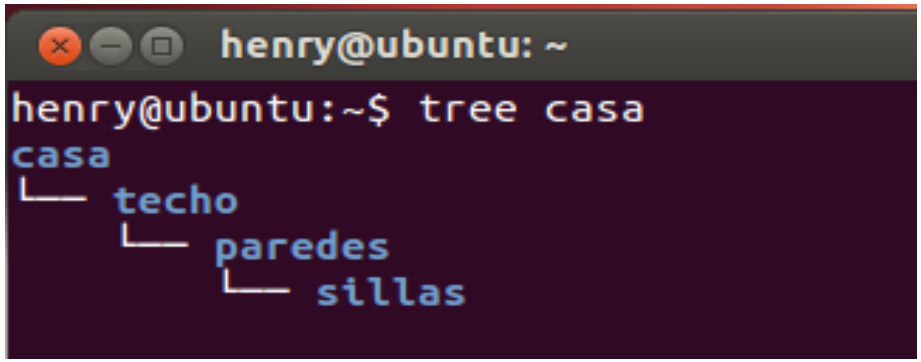
$ mkdir ejemplo # Crea el directorio **ejemplo** dentro del directorio actual

$ sudo mkdir /bin/quitar #Utilizamos ruta relativa y el comando sudo para crear dentro de **/bin** el directorio **quitar**

$ mkdir -p casa/techo/paredes/sillas # Con **-p** se crean los directorios intermedios.

$ mkdir cartas facturas datos #Crea los directorios: **cartas, facturas y datos** en el directorio actual.

NOTA: Para ver la estrutura completa de un directorio podremos utilizar el comado **tree** (aunque probablemente haya que instalarlo con **sudo apt-get install tree**). Veamos un **ejemplo**:



Estructura creada en el tercer ejemplo anterior

COMANDOS PARA FICHEROS Y DIRECTORIOS

COMANDO rmdir

rmdir se utiliza para eliminar directorios vacios. Si no están vacios primero habrá que eliminar los contenidos y luego el directorio.

Sintaxis: rmdir [directorio]

Ejemplos:

$ rmdir ejemplo # Elimina el directorio ejemplo

$ sudo rmdir /bin/quitar #Utilizamos sudo para eliminar el directorio quitar ubicado en el directorio bin.

COMANDO rm

rm se utiliza para borrar (desenlazar) ficheros o directorios

Sintaxis: rm [opciones] <ficheros | directorios>

Opciones básicas:

-r : borra recursivamente un directorio.

-f : borra forzosamente en caso de que no se tenga permiso de escritura en forma directa**.**

-i : ejecuta el comando de forma interactiva.

Ejemplos:

$ rm -r casa # Elimina recursivamente la estructura casa (creada anteriormente)

$ rm -ri casa #Elimina la estructura casa de forma recursiva y pidiendo confirmación.

COMANDOS PARA FICHEROS Y DIRECTORIOS

COMANDO chmod

chmod se utiliza para cambiar los permisos a directorios o archivos.
Sintaxis: chmod [opción][modo][directorio/archivo]

Opciones:

-c, --changes

Muestra un mensaje mencionando solamente aquellos ficheros cuyos permisos cambian realmente.

-f, --silent, --quiet

No muestra mensajes de error sobre ficheros cuyos permisos no se pueden cambiar.

-v, --verbose

Describe la acción efectuada o no para cada fichero.

-R, --recursive

Cambia recursivamente los permisos de directorios y sus contenidos.

COMANDOS PARA FICHEROS Y DIRECTORIOS

EJEMPLOS COMANDO chmod

Ejemplos:

$chmod 777 Prueba # Asigna a prueba los permisos rwx rwx rwx

$ sudo chmod 777 Prueba # Caso de que la carpeta pertezca a root se utiliza sudo.

#Otra forma de expresar los permisos sería indicando las letras asociadas, donde u es de usuario, g de grupo y o de otros. Ver el ejemplo siguiente:

$ chmod u=rwx,g=rw,o=r Prueba

 # Asignaría los permisos indicados tanto al directorio Prueba como a todo su contenido (herencia). Ver el siguiente ejemplo:

$ chmod -R 764 Prueba

COMANDOS PARA FICHEROS Y DIRECTORIOS

COMANDO chown

Utilizado para cambiar el propietario y el grupo de un fichero.
El dueño de un fichero solo lo puede cambiar el usuario root mientras que el grupo además de root, lo puede cambiar el propio dueño, siempre que pertenezca al nuevo grupo.

Sintaxis:
chown [opciones] <dueño>[.grupo] <ficheros>
chown [opciones] <grupo> <ficheros>
Opción: -R en los directorios cambia el dueño y/o el grupo recursivamente.

Ejemplos:
sudo chown pepe.pepe proyecto/
Otorga la propiedad del directorio proyecto/ al usuario pepe y al grupo pepe.

sudo chown -R mar.users /tmp/datos #otorga al usuario mar y al grupo users la propiedad de /tmp/datos.

sudo chgrp ftp /usr/ftp # otorga al grupo ftp la propiedad de /usr/ftp

COMANDOS PARA FICHEROS Y DIRECTORIOS

COMANDO cp (COPIANDO ARCHIVOS/DIRECTORIOS)

Sintaxis: **cp [opciones] fichero1 fichero2**

Copia el contenido de “fichero1” en “fichero2”.

Opciones:

-i Pide confirmación, antes de sobrescribirlo, si el fichero destino ya existe**.**

-r Copia un directorio junto con sus ficheros y subdirectorios en otro directorio. Si el directorio destino no existe, lo crea.

Ejemplos:

$ cp coches ejercicios/cars

Copia el fichero **coches**, en del directorio actual, al directorio **ejercicios**, con el nombre **cars**.

$ cp coches mbox ejercicios

Copia los archivos **coches** y **mbox** sobre el directorio **ejercicios**. Cuando se utilizan varios ficheros para copiar, el destino tiene que ser un directorio.

$cp –i coches ejercicios

Copia el archivo **coches** en el directorio **ejercicios**,pidiendo confirmación para sobrescribirlo, puesto que ya hay un archivo con igual nombre en el directorio **ejercicios**.

$ cp –r ejercicios so/pruebas

Copiamos el directorio **ejercicios** y todo su contenido al directorio **actividades**, en un nuevo directorio que creamos al hacer la copia, llamado **pruebas**.

COMANDOS PARA FICHEROS Y DIRECTORIOS

COMANDO mv (MOVER ARCHIVOS/DIRECTORIOS)

Utilizando **mv** para **renombrar** ficheros o directorios:

Sintaxis: mv [opciones] fichero1 fichero2
Sintaxis: mv [opciones] directorio1 nuevo_directorio

Utilizando **mv** para **mover** un fichero o el contenido de un directorio a otra ubicación:

Sintaxis: mv [opciones] fichero/s directorio
Sintaxis: mv [opciones] directorio1 directorio2

Opciones:

-i Caso de existir el fichero destino, pide confirmación antes de sobrescribirlo.
-f Si el fichero existe, lo sobrescribe sin pedir confirmación.

Ejemplos:

$ mv coches carros // Cambia el nombre del fichero **coches** por **carros**.
$ mv so sist_operativos **//** Cambia el nombre del directorio **so** por **sis_operativos**.
$ mv linux w7 S_operativos // Mueve los ficheros **linux** y **w7** al directorio **S_operativos**.
$ mv pruebas SO // Mueve los directorios y ficheros del directorio **pruebas** al directorio **SO**.

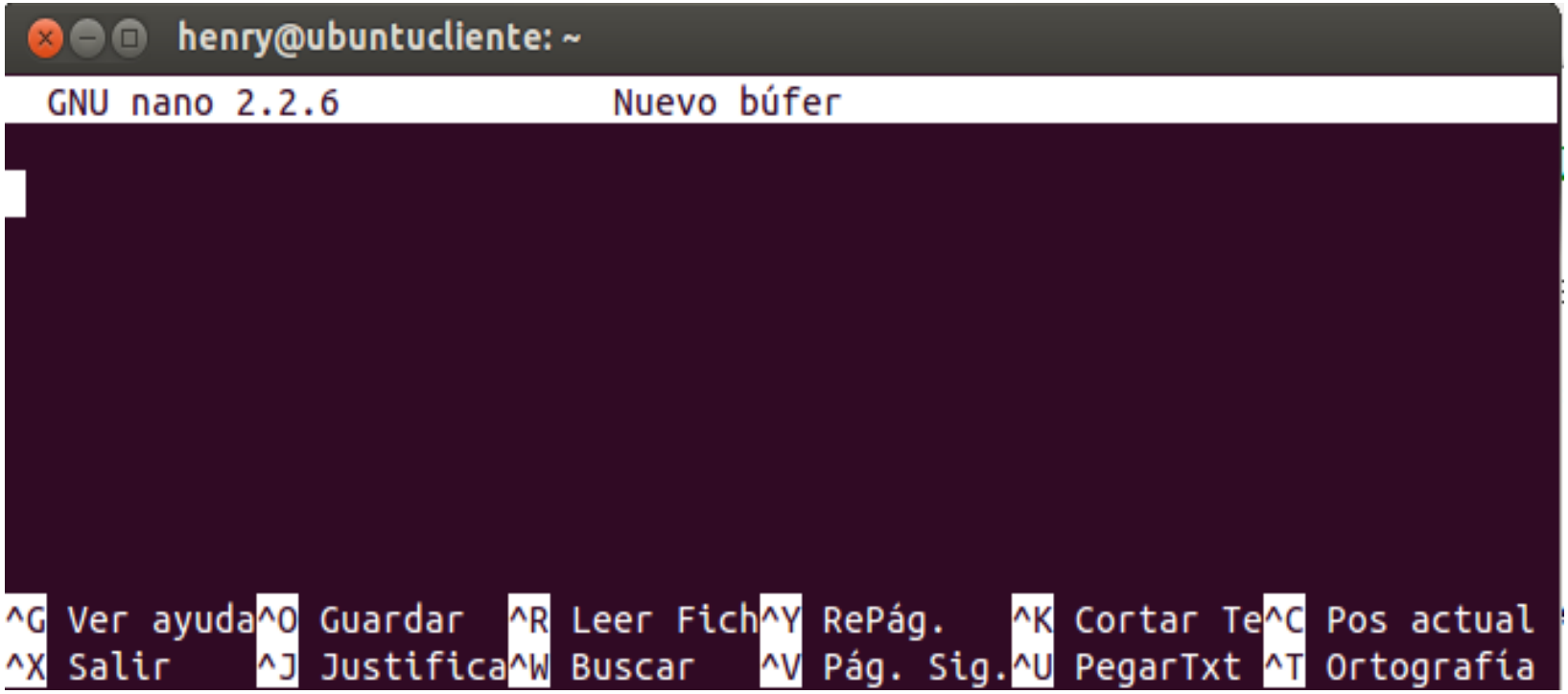
EDITORES, EN ENTORNO TEXTO, EN UBUNTU

Editor vi

Vi .- Editor de texto tradicionalmente utilizado por UNIX, cuyo manejo se basa en combinaciones de teclas.
Vim.- Es el editor vi mejorado que proporciona Ubuntu.

Editor nano

Se trata de otro editor que aparece en las distribuciones de Ubuntu y cuyo manejo resulta más sencillo que el **vi**



COMANDOS PARA VISUALIZAR Y EDITAR FICHEROS

COMANDOS more Y less

Muestran por pantalla uno o varios ficheros de forma paginada.
Cuando se alcanza el final del último fichero a paginar, **more** termina automáticamente, no así **less**.

Sintaxis: less [archivo]

Ejemplos:
$ less /etc/passwd
$ more /etc/passwd

Algunas teclas que podemos usar mientras usamos estos programas son:
q permite interrumpir el proceso y salir.
/p realiza búsquedas del patrón p dentro del texto. Para repetir la búsqueda del mismo patrón sólo es necesario escribir /.
[n]b en more permite regresar n páginas (por defecto n es 1).
[n]f en more se adelantan n páginas y en less, n líneas.

COMANDOS PARA VISUALIZAR Y EDITAR FICHEROS

COMANDO echo

Echo es un comando que nos permite escribir algo directamente por la salida estándar (la pantalla)

Sintaxis: **echo <texto> [argumentos]**

Algunos argumentos pueden ser:

- **-n** No agrega el carácter de fín de línea
- **\n** Inserta una nueva línea
- **\t** Inserta un tabulador horizontal
- **\v** Inserta un tabulador vertical
- **\r** Retorno de carro.

Ejemplos: **$ echo Hola Mundo**

COMANDOS PARA VISUALIZAR Y EDITAR FICHEROS

COMANDO cat

Lista el contenido de un fichero, pudiendo redirigir la salida hacia otro fichero (en lugar de la salida estandar: la pantalla).
Permite concatenar uno o mas archivos de texto.

Sintaxis: cat [ruta]<fichero>

Ejemplos:
$ cat archivo1.txt archivo2.txt //Concatena el contenido de archivo1.txt y archivo.2.txt mostrando el resultado por pantalla.

$ cat archivo1.txt archivo2.txt >contenido.txt //Concatena el contenido de archivo1.txt y archivo.2.txt guardando el resultado en el archivo contenido.txt.

$ cat >saludo.txt //Permite crear un documento de texto llamdo saludo.txt con lo que tecleemos en la consola hasta teclear Ctrl+C.

COMANDOS PARA HACER BÚSQUEDA EN FICHEROS I

COMANDOS grep

Devuelve todas las líneas que cumplen un patrón determinado en uno o varios ficheros.

Sintaxis: **grep [opciones] <patrón> [ficheros]**

Opciones:

-c devuelve sólo la cantidad de líneas que contienen al patrón.
-i ignora las diferencias entre mayúsculas y minúsculas.
-H imprime además de las líneas, el nombre del fichero donde se encontró el patrón. Es así por defecto cuando se hace la búsqueda en más de un fichero.
-l cuando son múltiples ficheros sólo muestra los nombres de aquellos donde se encontró al patrón y no las líneas correspondientes.
-v devuelve las líneas que no contienen el patrón.
-r busca en un directorio de forma recursiva.
-n imprime el número de cada línea que contiene al patrón.

Ejemplos:

$ grep linux /usr/share/doc
$ grep root /etc/passwd
$ grep -i pepe /etc/passwd
$ grep -c root /etc/group
$ grep -l -r -i img /var/www/html/
$ ls –lia | grep “carta roja”

Metacaracteres

Son caracteres que, en una expresión regular, son interpretados en su significado especial y no como los caracteres que normalmente representan.

Metacaracteres

Metacaracter	Explicación	Ejemplo
#	Comentario	#Esto es un comentario
“	Permite Sustitución	echo “Rutas $PATH”
‘’	Cadena Literal	‘Rutas $PATH’
$	Devuelve el contenido de una variable	$USER
*	0 ó más caracteres	chap*.ps
?	Un único carácter	lab?
[ ]	Conjunto de caracteres	[a-s] ó [1,5-9] ó [aeiou]
^	Signo de negación	[^3-8] ó [^aeiou]
;	Separar orden secuencial	echo $USER;ls
\	Nueva línea, para continuar orden en la siguiente. Escapar un metacaracter.	echo \ $USER [enter] echo *

Metacaracteres

- Además podemos usar las siguientes secuencias de escape (utilizando el carácter de “escape”: \) con los comandos **echo** y **sed**.

  - \n nueva línea
  - \t tabulador
  - \b espacio en blanco
  - \.
  - * (Así evitamos que * sea interpretado como un metacaracter)

Expresiones Regulares

Una Expresión Regular está formada por caracteres combinados con operadores.

Ciertos comandos como **grep** y **sed** permiten el uso de expresiones regulares para formar patrones de filtro / búsqueda

Expresiones Regulares

Nombre	Operador	Ejemplo	Significado
Alternativo*	|	x|y|z	x ó y ó z
Cualquier carácter, excepto salto de línea	.	L..a	Lana, Losa, Luna
Secuencia de Escape	\	*	*
Principio de línea	^	^x	Una línea que empieza por x
Fin de línea	$	x$	Una línea que acaba en x
Elemento precedente opcional*	?	xy?	x,xy
Repetición (0 ó más) del elemento precedente	*	xy*	x,xy,xyy,xyyy,…
Repetición (1 ó más) del elemento precedente*	+	xy+	xy,xyy,xyyy,…
Grupo*	()	(xy)+	xy,xyxy,xyxyxy,…
Conjunto	[]	[Hh]ola	Hola,hola

* Extendidas: Sólo disponibles con sed –r y grep -E

Expresiones Regulares

Nombre	Operador	Ejemplo	Significado
Cualquier carácter no contenido	[^]	[^A-KM-Z]ove	Love
El elemento precedente se repite n veces	{n}	[0-9]{5}	Para representar un número de 5 dígitos.
El elemento precedente se repite al menos n veces	{n,}	[0-9]{3,}	Representa un número con al menos 3 dígitos.
El elemento precedente se repite al menos n y no más de m veces	{n,m}	[a-zA-Z]{2,4}	Para representar una palabra que tiene entre dos y cuatro caracteres.

Actividad

- Qué hacen los siguientes comandos?.
  - mkdir dos dos3 dos4 dosa dosb; mkdir 3p 4p 5p
  - ls dos?
  - ls d[a-z]*
  - ls d[a-z]*[a,b]
  - ls d[a-z]*[ab]
  - ls d[a-z]*[0-9]
  - nano 3p/archivo (con el contenido que quieras)
  - ls [^4-5]p
  - ls [^4]p

Actividad

¿Qué hacen los siguientes comandos?

- cd 4p
- nano a1 (con el contenido que quieras)
- nano b2 (con el contenido que quieras)
- nano c3 (con el contenido que quieras)
- cat [a-z][0-9]
- cat [a-z][^12]
- cat [^a,c]?
- cat [^c]*

COMANDOS PARA HACER BÚSQUEDA EN FICHEROS

COMANDO find

Permite buscar de forma recursiva en un directorio todos los ficheros que cumplan ciertas condiciones. Las condiciones pueden estar relacionadas con el nombre de los ficheros, el tamaño, los permisos, el tipo, las fechas de acceso y modificación, etc.

Sintaxis: find [ruta desde donde se busca] [opciones de busqueda]

Opciones:
-name <expresión> permite especificar patrones para los nombres de los ficheros a buscar.
-iname <expresión> igual que el anterior, pero ignora mayúsculas/minúsculas
-type <tipo> permite indicar el tipo de fichero a buscar.
-size +/-<n> permite indicar el tamaño máximo y/o mínimo de los ficheros a buscar.
-perm [+|-]<modo> permite referirse a aquellos ficheros con unos permisos dados. El valor de **<modo>** se expresa en forma numérica.
-exec <comando> ; permite definir un comando a ejecutarse para cada resultado de la búsqueda.

NOTAS:
La cadena **{}** se sustituye por el nombre de los ficheros encontrados. El carácter; permite indicar la finalización del comando. (Tanto **{}** como **;** tienen que ir entre comillas o entre contrabarras para evitar que sea sustituido por el shell). Ver ejemplos en la siguiente diapositiva.

COMANDOS PARA HACER BÚSQUEDA EN FICHEROS

EJEMPLOS DEL COMANDO FIND

$ find /etc -name '*.conf' **#** busca en /etc todos los ficheros con extensión conf

$ find / -size +10240k -size -20480k # busca los ficheros cuyo tamaño esté entre 10M y 20M

$ find -perm +1000 -type d # busca los directorios que posean el permiso t

 # find / -name core -exec rm -i “{}” “;” # Busca todos los ficheros que se nombren core y los borra interactivamente.

COMANDOS PARA HACER FILTRADO DE FICHEROS

COMANDO stat

El comando stat muestra las características de un fichero. Por ejemplo: su nombre, permisos, tamaño en bytes, número del i-nodo que lo representa, las fechas de modificación y acceso, el tipo, el dueño, el grupo, etc.

Ejemplos: ***$ stat /etc/shadow***

COMANDOS PARA HACER FILTRADO DE FICHEROS

COMANDO sort

El comando sort ordena las líneas de un fichero mostrándolas por la salida estándar. De no especificarse un fichero toma la entrada estándar

Sintaxis: sort [opciones] [fichero]

Algunas opciones:
-r : ordena al revés.
-f : trata las mayúsculas y minúsculas por igual.

Ejemplo: **$ sort -f /etc/passwd**

COMANDOS PARA HACER FILTRADO DE FICHEROS

COMANDOS tail Y head

Los comandos **tail** y **head** muestran respectivamente el final y el comienzo (10 líneas por defecto) de uno o varios ficheros. De no especificarse al menos un fichero toman la entrada estándar.

Sintaxis: tail [opciones] [ficheros]
Sintaxis: head [opciones] [ficheros]

Algunas **opciones**:
-f para el caso de tail se continúa visualizando el final del fichero hasta que se interrumpa el proceso (Ctrl+C).
-q no coloca los encabezamientos con el nombre de los ficheros cuando se indican varios ficheros.
- <n> imprime las **n** últimas (tail) o primeras (head) líneas en lugar de las diez por defecto.

Ejemplos:
$ tail -f /var/log/messages
 # tail -20 /var/log/secure
 # head -50 /var/spool/mail/pepe
$ head -2 -q /etc/*.conf

COMANDOS PARA HACER FILTRADO DE FICHEROS

COMANDO wc

El comando wc (Word Count) imprime el número de líneas, palabras y bytes de uno o varios ficheros. Si son varios ficheros hace también un resumen de los totales. De no especificarse un fichero toma la entrada estándar.

Sintaxis: **wc [opciones] [ficheros]**

Algunas opciones:

-l sólo cuenta líneas.
-c sólo cuenta bytes.
-w sólo cuenta palabras.

Ejemplos:
$ wc -l /etc/passwd
$ wc -w /doc/dicciorario.txt

COMANDOS PARA HACER FILTRADO DE FICHEROS

COMANDO cut

Como su propio nombre indica, el comando **cut** tiene la característica de poder cortar caracteres y campos, con la posibilidad de usar delimitadores y otras opciones, para finalmente extraer las partes seleccionadas de cada fichero en la salida estándar.

El comando cut nos ofrece los siguientes **argumentos**:

-b, –bytes=LISTA muestra solamente estos bytes
-c, –characters=LISTA selecciona solamente estos caracteres
-d, –delimiter=DELIM usa DELIM en vez de caracteres de tabulación para delimitar los campos
-f, –fields=LISTA selecciona solamente estos campos; también muestra cualquier línea que no tenga un carácter delimitador, a menos que se especifique la opción -s

Ejemplos:

$ echo "Esto es una prueba, 1 2 3, probando" | cut -d "," -f 1
$ echo "Esto es una prueba, 1 2 3, probando" | cut -d "," -f 3
$ echo "Esto es una prueba, 1 2 3, probando" | cut -c2-4

COMANDOS PARA HACER BÚSQUEDA EN FICHEROS

COMANDO sed

En realidad **sed** no es un comando, sino un editor de textos que está diseñado para ser usado desde línea de comandos. Es muy útil cuando queremos modificar texto de forma no interactiva, veamos un ejemplo de uso de este editor:

$ echo “Hola mundo azul” | sed –e “s/mundo/amigo/”

-e indica que queremos ejecutar el siguiente comando
s es el comando de sustituir texto
/texto_antiguo/texto_nuevo/ es la forma de indicar qué queremos sustituir.
Algunos de los comandos que sed interpreta son los siguientes (hay bastantes más):
s sustituye texto
p imprime texto
d borra texto
n no muestra todas las líneas del fichero en la salida estandard

Ejemplos:

$ sed –e “1,10d” /home/joancadi/carta.txt
$ sed –e “1,10d” /home/joancadi/carta.txt > /home/joancadi/carta2.txt
$ sed –ne “4,20p” /etc/services nos muestra de la línea 4 a la línea 20 del fichero services.

COMANDOS PARA EMPAQUETAR Y DESEMPAQUETAR FICHEROS

COMANDO gzip Y gunzip

Los comandos gzip y gunzip permiten compactar y descompactar (comprimir y descoprimir) respectivamente uno o varios ficheros.

Sintaxis:

gzip [opciones] <ficheros/directorio>
gunzip [opciones] <ficheros/directorio>

Algunas opciones:

-r : dado un directorio comprime todos los ficheros presentes en él recursivamente.

-1 a -9 : especifica el grado de la compresión (-1 menor y más rápida -9 mayor y más lenta).

-S <sufijo> : permite especificar un sufijo o extensión para el fichero resultado (por defecto es gz).

Ejemplos:

$ gzip -9 big-file
$ gunzip big-file.gz
$ gzip -S .zip -r doc/
$ gunzip -S .zip -r doc/

EMPAQUETAR Y DESEMPAQUETAR FICHEROS

COMANDO tar

El comando tar (Tape Archiver) es una herramienta para agrupar varios ficheros aislados o el contenido de un directorio en otro fichero o dispositivo especial. El comando tar no comprime o compacta absolutamente nada, se limita a agrupar varios ficheros en uno solo, sin comprimirlos. Existe una opción (-z) que automáticamente ejecuta un gzip o un gunzip sobre el fichero agrupado.

Sintaxis: tar [opciones] <ficheros|directorios>

Algunas opciones:

-c permite crear (empaquetar).
-x permite extraer (desempaquetar).
-v activa el modo debug, donde se ven todos los mensajes.
-f <fichero> agrupa o desagrupa en o hacia un fichero y no utilizando la salida o entrada estándar como es por defecto.
-z compacta o descompacta el fichero resultante una vez agrupado o desagrupado con gzip y gunzip respectivamente.
-t lista el contenido de un fichero resultado de un agrupamiento.
-M agrupa en volúmenes.

Ejemplos:

```
$ tar cvfz /tmp/etc.tgz /etc
$ tar xvfz /tmp/etc.tgz
# tar cf uconf.tar passwd shadow groups
# tar xf uconf.tar
```

COMANDOS PARA COMUNICACIÓN ENTRE USUARIOS

COMANDO write

El comando **write** se utiliza para enviar un mensaje a un usuario conectado al sistema. Por defecto el mensaje se envía a la última terminal donde se haya conectado el usuario. Los usuarios pueden deshabilitar la posibilidad de recibir mensajes utilizando el comando mesg.

Sintaxis: write <usuario> [terminal]

Ejemplos:

***$ mesg y* # habilita la posibilidad de recibir mensajes**
$ write pepe tty3
Hola que tal
Ctrl-d

Si el usuario pepe está conectado a través de la terminal tty3 y tiene habilitada la posibilidad de recibir mensajes se mostrará en esta terminal:
Message from coco@deltha on tty2 at 16:35 ... (o el usuario que sea)
Hola que tal
EOF

$ mesg # muestra si está habilitada o no la posibilidad de recibir mensajes
$ mesg n # deshabilita la posibilidad de recibir mensajes

COMANDOS PARA COMUNICACIÓN ENTRE USUARIOS

COMANDO wall

El comando **wall** se emplea para enviar un mensaje a todos los usuarios conectados en el sistema que tengan habilitada la posibilidad de recibirlos (mesg y).

Ejemplos:
 # wall
Voy a apagar la máquina a
las 2:00 PM
El administrador
Ctrl-d

COMANDOS DE DESCONEXIÓN DEL SISTEMA

COMANDO exit

El comando exit permite terminar el shell actual. Si se tiene un único shell es equivalente a desconectarse del sistema, pero si se está en un subshell sólo se terminará este, retornando al shell anterior.

COMANDO logout

El comando logout permite desconectarse del sistema a partir de un login shell (primer shell que se ejecuta al establecer la conexión).
La secuencia de caracteres Ctrl-d permite terminar el shell actual. Si es un login shell equivaldrá a hacer logout y si no, a hacer exit.

COMANDO shutdown

El comando shutdown en realidad no sirve para desconectarse del sistema, sino para apagarlo totalmente.

Ejemplos:

 # shutdown –h now #Apaga el sistema (al igual que **halt**) ahora (**now**)

$ sudo shutdown -h 18:45 "El servidor se apagaeá para cambiar la tarjeta grafica"

$ sudo shutdown –r –g5 # El sistema se reiniciará (-r) en 5 minutos (-g5)

ADMINISTRACIÓN DE USUARIOS Y GRUPOS DEL SISTEMA

COMANDO adduser

El comando adduser permite añadir nuevos usuarios al sistema, además de establecer la información por defecto de los nuevos usuarios que se añadan.

NOTA: Para poder gestionar usuarios y grupos en el **entorno gráfico** de Ubuntu12.04 tenemos que instalar el siguiente paquete: **apt-get install gnome-system-tools**

Sintaxis: adduser [opciones] [login]

Ejemplos:

 # adduser pepe # crea el usuario pepe con las propiedades por defecto

 # adduser pedro amigos #añade a pedro (que ya existe) al grupo amigos existente.

 # adduser - -ingroup profesores juan #Crea un nuevo usuario y le asigna un grupo principal

COMANDO userdel

El comando userdel permite eliminar definitivamente un usuario del sistema.

Sintaxis: userdel [opciones] <login>

Ejemplos:

#userdel juan # elimina al usuario juan, pero mantiene su directorio personal

 # userdel -r pepe # elimina al usuario pepe y elimina su directorio personal.

ADMINISTRACIÓN DE USUARIOS Y GRUPOS DEL SISTEMA

COMANDO passwd

El comando **passwd** permite cambiar el password de un usuario. También puede bloquear, desbloquear y deshabilitar una cuenta. Si se invoca sin argumentos se asume el usuario actual.

Sintaxis: passwd [opciones] [login]

Ejemplos:

 # passwd pepe # asigna una contraseña para pepe

New UNIX password:
Retype new UNIX password:
passwd: all authentication tokens updated successfully

 # passwd -d pepe # deshabilita la cuenta del usuario pepe eliminando su password
 # passwd -l pepe # bloquea la cuenta del usuario pepe poniendo un signo ! delante de su password en el fichero /etc/shadow
 # passwd -u pepe # desbloquea la cuenta del usuario pepe

COMANDO usermod

El comando **usermod** se emplea para modificar algunas propiedades de los usuarios como: el login, el directorio base, el shell que se inicia al conectarse, los grupos a los que pertenece, la fecha de expiración de la cuenta, etc. También bloquea y desbloquea una cuenta.

Sintaxis: usermod [opciones] <login>
Ejemplos:
 # usermod -s /bin/csh pepe # coloca el shell **csh** para el usuario **pepe**
 # usermod -G users,disk pepe # señala como grupos secundarios de pepe a users y disk
 # usermod -e 2001-10-20 pepe # indica que la cuenta de pepe expirará el 20 de octubre del 2001

COMANDO chfn

El comando **chfn** permite cambiar la información de contacto de un usuario: el nombre completo, la oficina de trabajo y los teléfonos. Se almacena en **/etc/passwd.**

Sintaxis: chfn [opciones] [login]

Ejemplo:

 # chfn pepe

Changing finger information for pepe

Name [Pepito]: Pepon Pérez

Office [Recepción]: Redes

Office Phone []: 22-0909

Home Phone []: 44-3025

ADMINISTRACIÓN DE USUARIOS Y GRUPOS DEL SISTEMA

COMANDOS addgroup Y groupdel

Los comandos **addgroup** y **groupdel** añaden y eliminan un grupo en el sistema respectivamente.

Sintaxis: addgroup [opciones] <grupo>
groupdel <grupo>

Ejemplos:

 # addgroup equipo //Agrega un nuevo grupo, llamado equipo, al sistema
 # groupdel equipo //Elimina el grupo equipo del sistema

COMANDO groupmod

Permite modificar en un grupo su identificador y nombre.

Sintaxis: groupmod [opciones] <grupo>

Ejemplo:

 # groupmod -n usuarios users // cambia el nombre del grupo users por usuarios
 # groupmod -g 2000 equipo // **c**ambia el gid de equipo por el 2000

COMANDO gpasswd

El comando **gpasswd** permite administrar los grupos. Se puede utilizar para añadir y eliminar usuarios, señalar un administrador e indicar un password de grupo.
Sintaxis: gpasswd [opciones] <grupo>

Ejemplos:
 # gpasswd -A pepe admin # señala como administrador del grupo admin al usuario pepe
$ gpasswd admin # cambia el passwd del grupo admin
$ gpasswd -a joe admin # añade el usuario joe al grupo admin

COMANDO su

El comando su permite ejecutar un shell (u otro comando) cambiando los identificadores del grupo y del usuario actual. Si se le pasa **-** como primer argumento ejecuta el shell como un login shell, o sea se creará un proceso de login tal y como ocurre cuando un usuario se conecta al sistema. Si no se especifica el login del usuario se asume root.

Sintaxis: su [opciones] [login]

Ejemplos:

$ su -

$ su pepe //ejecuta un shell con el usuario pepe

$ su - -c "cat /etc/shadow" //ejecuta un comando con los privilegios de root

COMANDO sudo

Permite a los usuarios ejecutar programas con los privilegios de seguridad de otro usuario (normalmente el usuario root) de manera segura.
Los usuarios deben confirmar su identidad al ejecutar sudo dando **su propia contraseña** antes de ejecutar el programa requerido. Una vez se ha autenticado el usuario, y si el archivo de configuración **/etc/sudoers** permite dar al usuario acceso al comando requerido, el sistema lo ejecuta y lo registra. En un entorno gráfico, se usa **gksudo**.
El archivo de configuración **/etc/sudoers** especifica qué usuarios pueden ejecutar qué comandos en nombre de qué otros usuarios.

COMANDO id

El comando **id** muestra, dado un usuario, sus identificadores de usuario y de grupo principal (gid y uid) así como del resto de los grupos a los que pertenece. Sin argumentos se asume el usuario actual.

Sintaxis: id [opciones] [login]

Ejemplo:

 # id pepe uid=502(pepe) gid=502(pepe) groups=502(pepe),100(users)

MÁS COMANDOS

COMANDO tty

El comando tty imprime el dispositivo de carácter asociado a la terminal en la que se está trabajando.

Ejemplos:

***$ tty* /dev/tty2**

***$ tty* /dev/pts/0**

MÁS COMANDOS

COMANDO du

El comando du permite conocer la longitud (expresada en kilobytes por defecto) de una jerarquía de ficheros a partir de un directorio.

Sintaxis: du [opciones] [ficheros | directorios]

OPCIONES:

-h : (human readable view) imprime las unidades de la forma más representativa (Ej. M para megabytes).

-s : sumariza el tamaño de cada fichero/directorio sin profundizar recursivamente en estos últimos.

-c : produce un total cuando se utilizan varios argumentos.

Ejemplos:

$ du -h *

$ du -hsc /usr /home

MÁS COMANDOS

COMANDO who

El comando **who** muestra los usuarios conectados al sistema ya sea local o remotamente.

Sintaxis: who [opciones] [fichero] [am i]

Sin argumentos who muestra los logins de los usuarios conectados, el terminal en que se loguearon y la fecha y hora en que lo hicieron.

Algunas opciones:

-H : imprime un encabezamiento para las columnas.
-w : indica si está activada o no la posibilidad de recibir mensajes por parte de cada usuario conectado (+ indica que si, - que no y ?, desconocido).
-i : imprime además para cada usuario conectado que tiempo lleva sin interactuar con el sistema (idle time). Si lleva menos de un minuto pone un punto y si es más de 24 horas la cadena ``old".
-q : sólo muestra los login de los usuarios conectados y la cantidad total de ellos.

Ejemplos:

$ who
coco tty2 Oct 17 15:59
root tty3 Oct 17 12:40
alma :0 Oct 17 08:21
pepe pts/0 Oct 17 10:20
***$ who am I* gloin!pepe tty2 Nov 16 09:26**
***$ who –q* pepe root alma coco users=4**

MÁS COMANDOS

COMANDO w

El comando w muestra también los usuarios conectados al sistema además de lo que están haciendo (proceso que ejecutan en ese momento) y otras informaciones.

Sintaxis: w [opciones] [usuario]

Sin argumentos este comando muestra una primera línea con**:** la hora en que arrancó el sistema, cuanto tiempo lleva funcionando, cuántos usuarios hay conectados (sin incluir las sesiones gráficas) y tres porcientos de carga de la CPU: durante el último, los 5 y los 15 minutos anteriores. A continuación se muestra una tabla cuyas columnas representan: el login de cada usuario conectado, por que terminal lo ha hecho, desde que host, a que hora, el idle time exacto, la cantidad de segundos de CPU que han empleado todos los procesos que ha ejecutado ese usuario (JCPU) y el tiempo (PCPU) y nombre del comando que ejecuta actualmente.

Ejemplos:

$ w

$ w pepe

MÁS COMANDOS

COMANDO finger

El comando **finger** permite buscar y mostrar información asociada a los usuarios del sistema de acuerdo a sus nombres, apellidos o login. La información que muestra finger para cada usuario es:

El login.
El nombre y los apellidos.
El directorio base.
El shell.
La oficina y el teléfono.
El teléfono de la casa.
La lista de terminales a través de las que está conectado con la fecha, tiempo sin interactuar (iddle time) y si está deshabilitada la posibilidad de recibir mensajes.
La fecha y hora del último nuevo mensaje electrónico recibido y desde cuando no accede al buzón.
El contenido del fichero .plan en el directorio base.

Ejemplos:

$ finger alina
$ finger castellanos

MÁS COMANDOS

COMANDO ping
Permite comprobar si existe conexión con otro equipo o dispositivo conectado al red.

Sintaxis: ping [opciones] <ip_destino>

Algunas opciones:
-c <n> //envía n paquetes exactamente.
-i <n> //espera n segundos entre los envíos.
-s <n> //envía paquetes de n bytes. Se le suman los 8 bytes del header del paquete ICMP.
-q //sólo despliega el sumario final.

Ejemplos:

$ ping 10.0.0.8
$ ping www.google.es

MÁS COMANDOS

COMANDO ifconfig

El comando ifconfig permite configurar por parte de root las interfaces de red. Los usuarios distintos de root lo pueden invocar también con fines informativos. Para ello deben especificar el camino completo (/sbin/ifconfig) pues por defecto este no está en el path de los usuarios comunes. Sin argumento ifconfig despliega información acerca de la configuración y funcionamiento actuales de las interfaces de red activas.

ifconfig realmente modifica el fichero **/etc/networking/interfaces**. Este archivo se puede editar manualmente y para hacer efectivos los cambios tendremos que ejecutar: **/etc/init.d/networking restart.**

Ejemplos:

 # ifconfig eth1 192.168.1.1 up //asigna la dirección 192.168.1.1 al adaptador eth1 y la activa.
 # ifconfig eth0 down // permite "apagar" o bajar la interfaz de red eth0
 # ifconfig eth0 up //permite "encender" o levantar la interfaz de red eth0
 # ifconfig eth0 192.168.0.2 netmask 255.255.255.0 broadcast 192.168.0.255
//asigna una IP, máscara y dirección de difusión al adaptador eth0

MÁS COMANDOS

COMANDO apt-get

 # apt-get update //Actualiza los repositorios de Ubuntu.
 # apt-get install <nombre_paquete> //Instala el paquete indicado y sus dependencias.
 # apt-get remove <nombre_paquete> //Elimina el paquete indicado.
 # apt-get remove - - purge <nombre_paquete>
//Elimina el paquete indicado, incluyendo los archivos de configuración..
 # apt-get upgrade
//Actualiza el sistema operativo con las actualizaciones disponibles

MÁS COMANDOS

COMANDO aptitude

 # aptitude install <nombre_paquete>
//Instala el paquete especificado y sus dependencias

 # aptitude remove <paquete> //Elimina el paquete especificado y sus dependencias

 # aptitude update //Actualiza la lista de paquetes disponibles

 # aptitude upgrade
//Actualiza el sistema operativo con las actualizaciones disponibles

Diferencia entre apt-get y aptitude:

aptitude recuerda las dependencias que se han aplicado en la instalación de un paquete. Esto significa que si se instala o actualiza una aplicación con aptitude y luego se quiere desinstalar, aptitude borrará el programa junto con todas sus dependencias (excepto si son usadas por otros paquetes). Si se instala con apt-get o con entorno gráfico Synaptic, la desinstalación borrará sólo el paquete especificado, pero no las dependencias.

COMANDOS PARA INSTALAR PAQUETES. Parte I

INSTALACIÓN DE PAQUETES CON EXTENSIÓN .deb

Para instalar aplicaciones con extensión **.deb** sólo tienes que hacer doble click sobre el fichero en el navegador Nautilus y automáticamente se lanzará la aplicación **gdebi**, que se ocupará de instalar el paquete y buscar las dependencias que pudiera necesitar para su correcta instalación.

También se pueden instalar mediante la línea de comandos, con el comando **dpkg:**

Sintaxis: dpkg -i <nombre_del_paquete.deb>

En este caso también habrá que instalar manualmente las posibles dependencias del paquete.

DESINSTALACIÓN DE PAQUETES CON EXTENSIÓN .deb

El mismo comando también se puede usar para desinstalar el paquete:

Sintaxis: dpkg -r <nombre_del_paquete.deb>

COMANDOS PARA INSTALAR PAQUETES. Parte II

INSTALACIÓN DE PAQUETES CON EXTENSIÓN .bin

Para realizar esto mismo desde un terminal:

a) Damos permisos de ejecución.

```
$ sudo chmod +x [nombre_binario].bin
```

b) Instalamos el archivo binario.

```
$ sudo ./[nombre_binario].bin
```

COMANDOS PARA INSTALAR PAQUETES. Parte III

INSTALACIÓN DE PAQUETES CON EXTENSIÓN .run

Los archivos .run son asistentes, normalmente gráficos, que ayudan a la instalación. Para ejecutarlos basta con introducir en el terminal:

$ sh ./[archivo].run

Normalmente en el caso de que necesite permisos de superusuario (administrador) pedirá la contraseña; de no ser así basta con ejecutar en el terminal:

$ sudo sh ./[archivo].run

COMANDOS PARA INSTALAR PAQUETES. Parte III

INSTALACIÓN DE PAQUETES A PARTIR DEL CÓDIGO FUENTE (COMPILAR)

A veces se encuentran aplicaciones que no proporcionan paquetes de instalación, y hay que compilar a partir del código fuente. Para ello, lo primero que hay que hacer en Ubuntu es instalar un meta-paquete llamado build-essential. Desde el terminal ejecutamos los siguientes comandos:

$ sudo aptitude update
$ sudo aptitude install build-essential

COMANDOS PARA INSTALAR PAQUETES. Parte III

INSTALACIÓN DE PAQUETES A PARTIR DEL CÓDIGO FUENTE (COMPILAR)

En general, los pasos a seguir para compilar una aplicación son los siguientes:

1. Descargar el código fuente.
2. Descomprimir el código.
3. Entrar en el directorio creado al descomprimir el código.
4. Ejecutar el script configure (sirve para comprobar las características del sistema que afectan a la compilación, configurando la compilación según estos valores, y que crea el archivo makefile).
5. Ejecutar el comando **make**, encargado de la compilación.
6. Ejecutar el comando **sudo make install**, que instala la aplicación en el sistema, o mejor aún, instalar checkinstall (via aptitude), y ejecutar **sudo checkinstall**. Esta aplicación crea un paquete .deb de forma que no haya que compilarlos la próxima vez.

El uso de checkinstall tiene también la ventaja de que el sistema tendrá constancia de los programas instalados de esa forma, facilitando también su desinstalación.

CONVERTIR FICHEROS .rpm A .deb

Algunas distribuciones de Linux, como por ejemplo Red-Hat y Mandriva, usan paquetes .rpm, organizados de manera diferente a los paquetes .deb de Debian y Ubuntu.

Para instalar estos paquetes es preciso convertirlos antes al formato .deb. Para ello se usa la aplicación **alien**. Se puede instalar via Synaptic o aptitude.

Uso:

Abre una terminal (**Aplicaciones** → **Accesorios** → **Terminal**) y ejecutar la siguiente instrucción:

$ sudo alien nombre_del_paquete.rpm

De esta forma el programa crea un fichero con el nombre del paquete, pero con extensión **.deb**, que se podrá instalar siguiendo la explicación **Paquetes con extensión .deb.**

NOTA

A lo largo del curso veremos más comandos incluidos en cada bloque correspondiente.